

CSED441 Intro. to Computer Vision : Assn2 Report

November 2, 2025

Student 20240505 Hyunseong Kong

1. Overview

The second assignment is to implement HOG and linear SVM to detect pedestrians from the given image dataset.

2. Discussion

2-1. SVM Hyperparameters

Question 1. Experiment with different SVM hyperparameters and report their impact on classification performance.

Answer 1 (C). 기본값: 1.0

이 파라미터는 가중치 갱신을 위한 `compute_gradient`와 loss를 계산하기 위한 `compute_hinge_loss`에서 사용된다. linear SVM에서 데이터가 완전히 분리 불가능한 경우에, C 값이 클수록 오분류를 최소화하는 것에 모델이 집중하게 된다. 반면, C가 낮으면 오분류에는 큰 관심이 없고, margin을 최대화하는 것에 모델이 집중하게 된다. 즉, C 값이 크면 과적합의 위험이 생기는 반면, C 값이 작으면 과소적합의 위험이 생긴다.

C	Final Loss	Accuracy(%)
0.0	0.0	53.4
0.1	66.68	96.43
0.5	160.75	97.06
1.0	245.62	97.48
3.0	329.15	96.75
10.0	1605.35	97.06

위의 표를 보면, C가 없을 때는 Loss 자체는 적지만 정확도가 현저히 떨어지며(과소적합), C를 늘려갈수록 최종 Loss 자체는 늘어나지만 정확도가 점점 올라가는 것을 확인할 수 있다. 이때, C가 1.0에서 정확도가 97.48%로 정점을 찍은 후, C=3.0 (96.75%), C=10.0 (97.06%)에서는 오히려 정확도가 소폭 하락하거나 정체된다. 이는 C값이 일정 수준(1.0)을 넘어가자, 훈련 데이터에 너무 집중하여(과적합) 새로운 데이터에 대한 일반화 성능이 미세하게 저하될 수 있다는 것을 설명하는 하나의 근거이다.

Answer 2 (learning_rate). 기본값: 0.001

이 파라미터는 `compute_gradient`로 계산된 가중치를 이용한 실제 가중치 갱신을 수행하는 `LinearSVM.train`에서 사용된다. learning_rate가 클수록 한번에 크게 가중치를 업데이트하게 되며, 반면 작으면 가중치를 많이 업데이트하지 않는다.

즉, learning_rate 값이 크면 모델이 빠르게 최솟값을 향해 수렴하고 local minimum에 갇힐 가능성이 줄어들게 되나 동시에 불안정하게 수렴하며 최솟값에 완전히 접근하기는 어려워지고, learning_rate 값이 작으면 모델이 최솟값을 향해 수렴하는 속도가 느리고 local minimum에 갇힐 가능성도 생기게 되나 수렴이 랜덤하게 선택된 배치에 비교적 무관하며, 안정적이다.

learning_rate	Final Loss	Accuracy(%)
0.0001	263.77	96.85
0.001	263.77	96.85
0.01	316.37	96.22
0.1	4856.21	83.42

위의 표를 보면, learning_rate가 늘어나면 최종 Loss가 점점 증가하는 것을 볼 수 있다. 정확도는 일정하게 유지되다가 learning_rate가 너무 크면(0.1) 오히려 감소하며 동시에 Loss는 폭증하게 되는데, 이는 학습률이 너무 클 경우, 모델이 최적의 가중치를 찾지 못하고 불안정하게 수렴하며 최솟값에 접근하기 어려워진다는 현상을 보여주는 증거이다.

Answer 3 (epochs). 기본값: 100

이 파라미터는 가중치 갱신을 수행하는 `LinearSVM.train`에서 사용된다. 1 epochs는 데이터 전체를 한번씩 이용해 모델을 학습시키는 것을 의미한다. 즉, 100 epochs는 모델을 학습할 때 전체 데이터셋을 100회 이용했다는 의미이다. 즉, epochs 값이 크면 모델을 더 많이 학습시키기 때문에 모델이 보다 잘 수렴하게 되나 시간이 오래 걸리게 된다. 또한, epochs 값이 클수록 과적합의 위험이 커진다. 반면, epochs 값이 작으면 모델이 충분히 학습되지 않아 수렴이 덜 된 상태가 된다.

epochs	Final Loss	Accuracy(%)
1	616.72	92.44
2	442.95	97.06
10	273.61	96.12
100	245.62	97.48
500	229.50	97.38

epochs=1일 때는 정확도가 92.44%, Final Loss가 616.72로 매우 높아, "수렴이 덜 된 상태"임이 분명하다. 반면, epochs=2만 되어도 정확도가 97.06%로 크게 상승하면서 모델이 빠르게 수렴하는 것을 확인할 수 있다. 이러한 정확도는 epochs=100일 때 97.48%로 정점을 찍고, epochs=500에서는 Final Loss 자체는 229.50으로 감소했지만, 정확도는 97.38%로 epochs=100일 때보다 미세하게 하락하는 것을 확인할 수 있다. Final Loss 자체는 계속 줄어들지만 Accuracy가 하락하는 현상은 epochs 값이 클수록 과적합의 위험이 커진다는 것을 입증한다.

Answer 4 (batch_size). 기본값: 32

이 파라미터는 가중치 갱신을 수행하는 `LinearSVM.train`에서 사용된다. 데이터 전체를 이용해 가중치 갱신을 계산하는 것이 이상적이지만, 성능상의 문제가 있기 때문에 과제에서는 SGD를 이용해 가중치 갱신을 수행한다. 이때, 1번에 랜덤하게 선택할 데이터의 개수를 결정하는 것이 이 파라미터의 역할이다.

즉, batch_size 값이 크면 한 번의 가중치 갱신이 오래 걸리지만 epoch 당 갱신은 적게 걸린다. 또한, 경사 추정이 더 정확하게 수행되지만, 한번에 이용하는 메모리가 늘어나게 된다. 게다가, 이렇게 계산된 경사도가 너무 정확하면, 손실 함수의 local minimum 영역에서 벗어나기 어렵다. 반면, batch_size 값이 작으면 수렴은 불안정하고 전체 학습 속도가 느리지만 메모리를 적게 사용한다는 장점이 있다. 다만, batch_size는 epochs랑 연결되어 batch_size를 너무 키우면 epochs당 수행하는 가중치 갱신 횟수가 줄어들기에 epochs 수도 함께 늘려야 batch_size를 늘렸을 때의 장점이 드러난다.

batch_size	Final Loss	Accuracy(%)	Time Taken
1	1078.06	91.61	00:11
4	802.44	95.28	00:06
8	433.14	96.54	00:02
32	245.62	97.48	00:02

batch size가 1과 같이 크기가 작은 경우, 정확도(91.61%)가 낮다. 이는 수렴이 불안정하다는 것에 대한 하나의 증거이다. 반면, 배치 크기가 커질수록 경사 추정이 정확해지며, batch size가 32일 때 97.48%로 가장 높은 정확도를 보여준다. 반면, 학습 시간은 batch size가 1일 때 11초로 가장 느렸고, 배치 크기가 커질수록 6초, 2초로 시간이 크게 단축되는 것을 확인할 수 있다. 이는 연산 효율이 증가하여 에포크당 총 학습 시간이 줄어들었기 때문이다.

2-2. Sliding Window Detection Parameters

Question 2. Experiment with sliding window detection parameters and analyze the balance between computational efficiency and detection quality.

Answer 5 (scale_factor). 기본값: 1.2

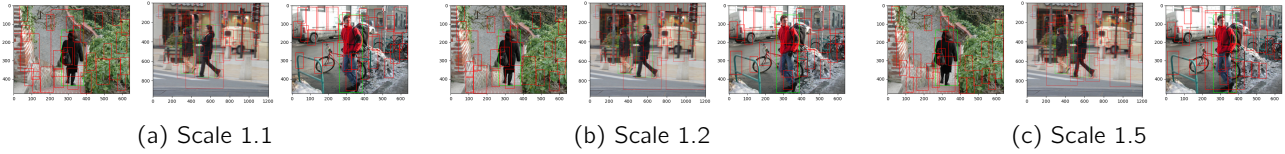
이 파라미터는 이미지 피라미드를 만드는 `generate_pyramid`에서 사용한다. 우선, 우리의 모델은 고정된 크기의 윈도우를 이용하기 때문에, 이미지 피라미드를 사용해 다양한 크기의 객체를 검출한다. scale_factor는 피라미드의 인접한 두 이미지 사이의 크기 비율을 결정한다. 피라미드를 만드는데는 max_scale, min_size도 영향을 주는데, 두 파라미터가 각각 최대 이미지 스케일, 최소 이미지 스케일 크기를 결정하기 때문이다.

즉, scale_factor가 크면 피라미드의 이미지 크기가 보다 듬성하게 만들어져 다양한 크기의 객체를 검출하는 성능은 떨어지지만, 계산 비용은 줄어든다. scale_factor가 작으면 피라미드가 촘촘하게 만들어져 다양한 크기의 객체를 모두 검출할 수 있지만, 계산 비용이 증가한다.

scale_factor	Time Taken
1.5	00:26 ; 01:33 ; 00:18
1.2	00:35 ; 02:14 ; 00:28
1.1	00:53 ; 04:09 ; 00:51

첫번째 이미지의 경우, $scale_factor=1.5$ (등성한 피라미드)일 때 첫 이미지 처리 시간이 26초대였던 것이, $scale_factor=1.2$ (중간 정도)일 때 35초대로, $scale_factor=1.1$ (촘촘함)일 때 53초대로 급격히 증가했다. 다른 이미지 또한 비슷한 양상을 보인다. 이는 $scale_factor$ 가 작아져 피라미드 층이 촘촘하게 만들어질수록, 계산 비용이 증가한다는 것에 대한 증거이다.

또한, 아래의 이미지를 통해 Scale 값이 증가할수록 실제 만들어지는 박스의 수가 줄어드는 것을 확인할 수 있다.



Answer 6 (step_size). 기본값: 8

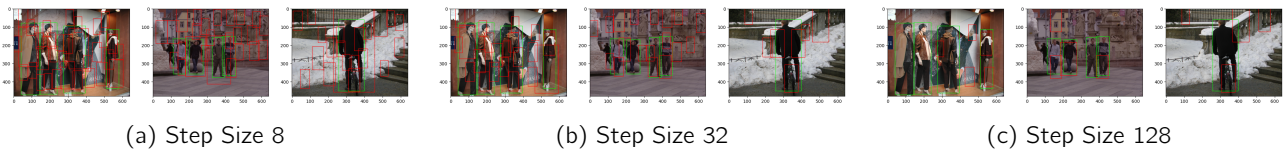
이 파라미터는 이미지 위의 슬라이딩 윈도우를 만드는 `sliding_window`에서 사용한다. 우선, 모델이 하고자 하는 것은 보행자를 검출하는 것이며, 이미지 위의 모든 부분을 순회하면서 보행자 후보를 찾는다. 이에 앞에서 만든 이미지 피라미드의 각 이미지에서 고정된 크기의 윈도우를 이용해 이미지의 일부분을 자르게 되는데, 이 파라미터는 이러한 윈도우가 움직이는 정도를 결정한다.

즉, $step_size$ 가 크면 슬라이딩 윈도우가 한번에 큰 보폭으로 이동하면서 보행자 후보군이 적게 만들어지고 모델의 속도는 증가하지만 보행자를 검출하는 성능은 떨어진다. 반면, $step_size$ 가 작으면 윈도우가 조금씩 움직이면서 매우 많은 후보군을 만들고 이에 모델의 속도가 감소하지만 이미지의 보행자를 잘 검출할 수 있게 된다.

step_size	Time Taken
8	00:30 ; 00:30 ; 00:30
32	00:03 ; 00:03 ; 00:02
128	<00:01 ; <00:01 ; <00:01

첫번째 이미지의 경우, $step_size=8$ (촘촘함)일 때 30초가 걸리던 작업이, $step_size=32$ (중간 정도)일 때 3초로, $step_size=128$ (등성함)일 때는 1초로 처리 시간이 기하급수적으로 감소했다. 다른 이미지 또한 비슷한 양상을 보인다. 이는 $step_size$ 가 커질수록 슬라이딩 윈도우가 한번에 큰 보폭으로 이동하므로 모델의 속도가 증가한다는 것에 대한 증거이다.

또한, 아래의 이미지는 $step_size$ 가 커질수록 box가 덜 생긴다는 것을 입증한다.

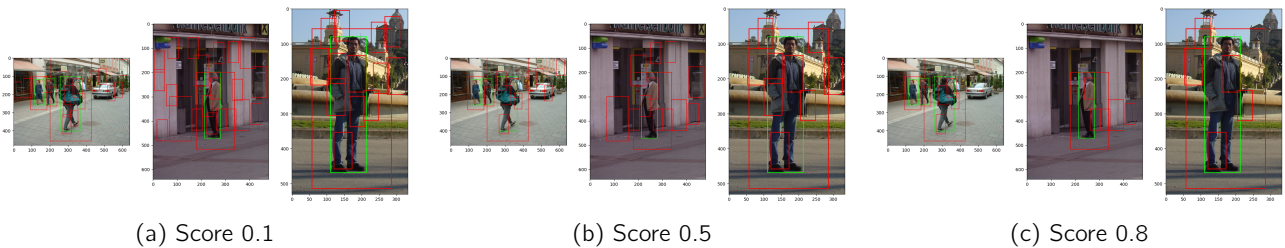


Answer 7 (score_threshold). 기본값: 0.2

이 파라미터는 완성된 HOG와 Linear SVM 모델을 이용해 주어진 후보군이 실제로 보행자인지 판별하는 `Object-Detector.detect`에서 사용한다. HOG로 특징을 추출한 후 이를 SVM의 가중치로 계산하면 점수가 나오게 된다. 이 점수가 높을수록(1에 가까울수록) 보행자에 가깝다는 의미이며, 보행자의 경계를 결정짓는 기준이 $score_threshold$ 이다.

즉, $score_threshold$ 가 높을수록 보행자로 탐지되는 후보군의 개수는 줄어들지만 보행자로 검출된 후보군의 신뢰도는 올라가고, $score_threshold$ 가 낮을수록 보다 많은 후보군이 보행자로 탐지되지만 후보군의 신뢰도는 내려간다.

아래의 이미지는 이를 검증한다.



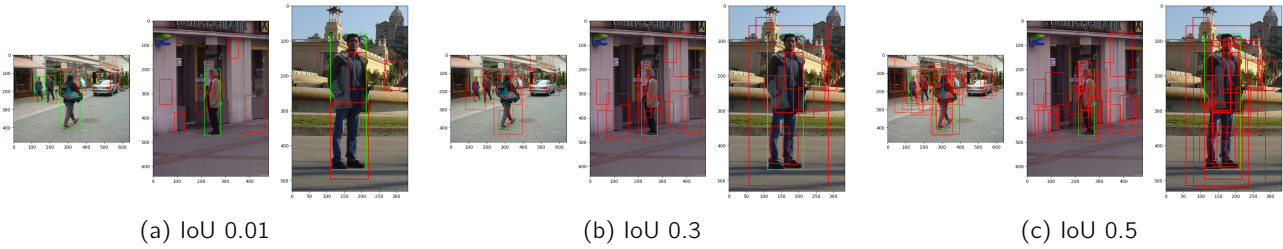
Answer 8 (iou_threshold). 기본값: 0.1

이 파라미터는 중복된 검출을 삭제하여 가장 신뢰도가 높은 결과만을 남기는 `non_maximum_suppression`에서 사용한다. 우선, 모델이 위의 슬라이딩 윈도우를 통해 만든 보행자 후보군은 상당수가 겹치게 된다. 이는 윈도우 크기인 (64, 128)에 비해, $step_size$ 는 8로 매우 작기 때문이다(기본값에서의 비교). 여기서 $iou_threshold$ 는 score가 높은 결과부터 순회하며, 그 윈도우가 score가 낮은 윈도우와 일정 수 이상 겹치면 score가 상대적으로 낮은 검출을 삭제할

때, 겹치는 정도를 결정하는 파라미터로 사용된다.

즉, `iou_threshold`가 크면 겹치는 영역이 아주 크지 않는 이상 최종 후보군에 모두 들어가게 된다. 반면 `iou_threshold`가 작으면 겹치는 영역이 조금만 있어도 `score`를 기준으로 삭제되어 최종 결과에는 보이지 않게 된다.

아래의 이미지는 이를 검증한다.



Note: Because of the performance issue, the `step_size` is set to 16 while modifying the parameter '`iou_threshold`'.

2-3. SVM Classifier Performance

Question 3. Your SVM classifier likely achieves decent performance on cropped patches, but the sliding window detector's performance on full images may be disappointing. Analyze this gap and propose potential approaches to improve detection performance.

Answer 9. 우선 negative sample이 부족하다. SVM 훈련 시 사용한 음성 샘플이 하늘, 잔디밭 등의 명확한 배경에서 가져온 쉬운 샘플이며, 실제 이미지에는 보행자와 비슷해 보이는 복잡한 배경이 많아 검출기가 실제 이미지에서 이들을 보행자로 오인하고 오탐지한다.

또한, 슬라이딩 윈도우와 이미지 피라미드를 사용해도 검출 윈도우 크기(64×128)와 실제 보행자의 크기가 정확히 일치하지 않을 수 있다. 이 경우, 특히 먼 거리나 너무 가까이 있는 보행자는 특징 추출이 불안정해진다. 게다가, 우리가 사용하는 윈도우의 종횡비는 고정되어 있는데 이것이 학습 데이터와 달라도 학습에 문제가 발생한다.

이를 해결하기 위해, 우선 어려운 negative sample을 추가적으로 학습해야 한다. 이미 완성된 SVM 분류기를 사용해 보행자가 없는 배경 이미지를 스캔하고, 분류기가 보행자라고 오탐지한 부분을 찾는다. 이러한 오탐지 영역들(즉, SVM이 제대로 탐지하지 못하는 어려운 데이터)을 새로운 negative sample을 분류한 뒤, 원래의 훈련 데이터셋에 추가한다. 또한, 이미지 피라미드의 scale 파라미터를 더 작게 설정해 피라미드 층을 촘촘하게 생성한다. 이렇게 하면 다양한 크기에 대한 민감도가 높아져 검출률이 높아진다. 비슷하게, 슬라이딩 윈도우의 간격을 줄여 스캔 자체를 촘촘하게 해도 작은 물체나 윈도우 경계에 걸쳐 있는 물체를 놓칠 확률이 줄어든다.